CSE 211 — Theory of Computation

Introduction · Simplifying CFGs · Chomsky Normal Form · CYK

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Atif Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — a 15-mark CNF or CYK question in every recent paper
Lesson 1 — The Three Areas and the Course Plan — automata, computability, complexity; models of computation
Lesson 2 — Simplifying Grammars — useless symbols, ε-productions, unit productions
Lesson 3 — Converting to Chomsky Normal Form — the 4-step order, terminals → variables, breaking long bodies
Lesson 4 — The CYK Membership Algorithm — triangular table, X_{ij}, worked tables
Quick Notes — Intro & CNF — revision bullets + key diagrams

> **NOTE** Slide numbers refer to the PDF pages of Lecture - 1 (intro_2).pdf (14 pages) and Lecture - 2 (cnf).pdf (35 pages). Atif Sir follows Hopcroft–Motwani–Ullman. Every CNF conversion was checked by a program (same language as the original grammar on all strings up to length 7–8) and every CYK table was generated by a program. Practice provenance: Actual = taken from a TF paper; Adapted = TF question with changed details; Constructed = new question in TF style.

Exam Map — What the TF Papers Ask

Session	CNF / CYK questions
2023-24	Q6 CYK: is 0010 in L(G) (S→BZ|CY|ZZ|YY …)? Describe L(G) (15)
2022-23	Q7 CYK: is 0100 in L(G), S→AB|AY|YA|1, A→ZA|YA|0|1, B→YA, Z→0, Y→1? (15)
2021-22	Q5 Convert S→A1A|B, A→0A|1A|ε, B→1 to CNF (15)
2020 (Jan term)	Q7 CYK: is 10011 in L(G), S→BZ|CY|ZZ|YY, B→ZA, C→YA, A→ZA|YA|0|1, Z→0, Y→1? (15)
2019-20	Q7 CYK: is 01100 in L(G) (S→BZ|CY|ZZ|YY …)? Describe L(G) (15)
2017-18	Q7(a) Convert S→A1A|B, A→0A|1A|ε, D→00 to CNF (8) · Q7(b) CYK: is 01101 in the language of the CNF grammar of 7(a)? (7)
2016-17	Q6(b) Convert A→BAB|B|ε, B→00|ε to CNF (10)
2014-15	Q5(c) CNF definition; convert A→BAB|B|ε, B→00|ε to CNF (9⅓)
2008-09	Q2(a) S→ASB|ε, A→aAS|a, B→SbS|A|bb: eliminate ε-productions (7⅓) · Q2(b) Same grammar: eliminate unit productions (8) · Q2(c) Same grammar: eliminate useless symbols (8)
2007-08	Q13 CNF definition; CYK for 'abaab' with S→AB|BC, A→BA|a, B→CC|b, C→AB|a (10)

EXAM FOCUS A full 15-mark CNF or CYK question in every paper since 2017-18 (2023-24, 2022-23, 2021-22, 2019-20, Jan-2020, 2017-18). The CYK grammar S → BZ | CY | ZZ | YY … was set three times (2023-24, 2019-20, Jan-2020) with a different string — learn Lesson 4's table method and the language of that grammar. CNF conversion of S → A1A | B, A → 0A | 1A | ε came twice (2021-22, 2017-18) and A → BAB | B | ε, B → 00 | ε twice (2016-17, 2014-15).

Lesson 1 — The Three Areas and the Course Plan

Lecture 1, slides 1–14

Theory of computation has three areas linked by one question — *what are the fundamental capabilities and limitations of computers?* (sl. 2–3): **automata** (DFA, NFA, CFG, PDA — mathematical models of machines), **computability** (decidability: what can or cannot be solved), **complexity** (tractability: what can or cannot be solved efficiently — P, NP, NP-complete, NP-hard, PSPACE). 'Computer science is no more about computers than astronomy is about telescopes' (Dijkstra, sl. 4); computability is like asking whether we can exceed the speed of light rather than how to build faster vehicles (sl. 5).

Model	Memory	Power / use (sl. 6, 11–13)
Finite automata (DFA, NFA)	limited — finite states	regular languages; compilers (lexers), hardware control units; cannot do 0^n1^n
Context-free grammars / PDA	a stack	context-free languages; parsers, AI, linguistics
Turing machine	unbounded tape	can simulate a computer; what a TM cannot solve is beyond computation

Alan Turing proposed the Turing machine, helped break Enigma and proposed the Turing test (sl. 7). Syllabus (sl. 8–10): regular languages, context-free languages, Turing machines and their equivalence with a computer, decidability, time and space complexity. Textbook: Hopcroft, Motwani & Ullman; references Sipser and Lewis & Papadimitriou (sl. 14).

> **NOTE** No TF question has come from this lecture. The ToC-in-the-world questions (passwords, cryptography, hard problems) are answered in Hakim Sir's Intro note, Lesson 1.

Lesson 2 — Simplifying Grammars

Lecture 2, slides 19–30

1. Useless Symbols (sl. 22–23)

X is **generating** if X ⇒∗ w for some terminal string w (every terminal is generating). X is **reachable** if S ⇒∗ αXβ. X is **useful** if it is both, i.e. it appears in some derivation of a terminal string from S; otherwise useless. Find generating symbols inductively (terminals; then any A with a body of generating symbols), delete the non-generating ones, THEN find reachable symbols from S and delete the rest.

```
S → AB | a,  A → b       B is not generating → delete S → AB:  S → a,  A → b
                         A is not reachable  → delete A:        S → a          (sl. 23)
```

TRAP Order matters: generating first, then reachable. In the other order a symbol made unreachable only by deleting a non-generating production would survive (here A).

2. ε-Productions (sl. 24–26)

A is **nullable** if A ⇒∗ ε (basis: A → ε; induction: A → C1…Ck with all Ci nullable). For every production with nullable symbols in its body, add all versions with any subset of those symbols removed (but never add A → ε). If the language contains ε, keep S → ε (Hopcroft: the grammar is then 'CNF + S → ε').

```
S → AB,  A → aAA | ε,  B → bBB | ε          nullable: A, B, S
S → AB | A | B,  A → aAA | aA | a,  B → bBB | bB | b,   and add S → ε (S nullable)   (sl. 25–26)
```

3. Unit Productions (sl. 27–30)

A unit production is A → B (both variables). (A, B) is a **unit pair** if A ⇒∗ B using unit productions only ((A, A) by zero steps). For each unit pair (A, B) add A → α for every non-unit production B → α; then drop all unit productions.

- The productions to be added/kept

Pair	Productions
(E, E)	$E \to E + T$
(E, T)	$E \to T * F$
(E, F)	$E \to (E)$
(E, I)	$E \to a \mid b \mid Ia \mid Ib \mid I0 \mid I1$
(T, T)	$T \to T * F$
(T, F)	$T \to (E)$
(T, I)	$T \to a \mid b \mid Ia \mid Ib \mid I0 \mid I1$
(F, F)	$F \to (E)$
(F, I)	$F \to a \mid b \mid Ia \mid Ib \mid I0 \mid I1$
(I, I)	$I \to a \mid b \mid Ia \mid Ib \mid I0 \mid I1$

Figure 2.1 — Unit pairs and the productions kept for E/T/F/I (sl. 29)

Exam Approach

1. List nullable variables; rewrite every production (all subsets); note whether ε ∈ L.
2. List unit pairs (closure); replace unit productions.
3. Remove non-generating, then unreachable symbols.

4. Show the grammar after each step — the marks are per step.

Practice — Lesson 2

(a)–(c) Actual — 2008-09 Q2(a)–(c).

(a) For S → ASB | ε, A → aAS | a, B → SbS | A | bb eliminate ε-productions. (7⅓)

(b) Eliminate unit productions. (8)

(c) Eliminate useless symbols. (8)

✓ Solution

(a) Nullable: S only (A and B always produce a terminal). Remove S from bodies in all ways:

```
S → ASB | AB
A → aAS | aA | a
B → SbS | Sb | bS | b | A | bb          (ε ∈ L, so the language is L(G′) ∪ {ε}; keep S → ε
separately if required)
```

(b) Unit pairs: (S,S), (A,A), (B,B), (B,A). Replace B → A by A's non-unit bodies:

```
S → ASB | AB
A → aAS | aA | a
B → SbS | Sb | bS | b | bb | aAS | aA | a
```

(c) Generating: a, b; A (A → a); B (B → b); S (S → AB). All generating. Reachable from S: A, B (and a, b). No useless symbols — the grammar of (b) is final. (Both steps were checked: the language is unchanged on all strings up to length 6.)

Lesson 3 — Converting to Chomsky Normal Form

Lecture 2, slides 2–3, 19–21, 31–35

MEMORIZE CNF: every production is A → BC (two variables) or A → a (one terminal), and there are no useless symbols (sl. 2). Every non-empty CFL without ε has a CNF grammar; with ε, add S → ε (S not on any right side). Order (sl. 21): (1) eliminate ε-productions, (2) eliminate unit productions, (3) eliminate useless symbols, (4a) put terminals of long bodies into new variables (A → a), (4b) break bodies of length ≥ 3 into chains.

1. Steps 4a and 4b — Worked Example (sl. 32–35)

- We introduce these productions, and replace every terminal in a body that is other than a single terminal by the corresponding variable

$$
\begin{array}{lcl}
E & \to & EPT \mid TMF \mid LER \mid a \mid b \mid IA \mid IB \mid IZ \mid IO \\
T & \to & TMF \mid LER \mid a \mid b \mid IA \mid IB \mid IZ \mid IO \\
F & \to & LER \mid a \mid b \mid IA \mid IB \mid IZ \mid IO \\
I & \to & a \mid b \mid IA \mid IB \mid IZ \mid IO \\
A & \to & a \\
B & \to & b \\
Z & \to & 0 \\
O & \to & 1 \\
P & \to & + \\
M & \to & * \\
L & \to & (\\
R & \to &)
\end{array}
$$

Figure 3.1 — Replacing terminals in long bodies by new variables (sl. 34)

- Introduce variables to break bodies of length 3 or more

$$
\begin{array}{lcl}
E & \to & EC_1 \mid TC_2 \mid LC_3 \mid a \mid b \mid IA \mid IB \mid IZ \mid IO \\
T & \to & TC_2 \mid LC_3 \mid a \mid b \mid IA \mid IB \mid IZ \mid IO \\
F & \to & LC_3 \mid a \mid b \mid IA \mid IB \mid IZ \mid IO \\
I & \to & a \mid b \mid IA \mid IB \mid IZ \mid IO \\
A & \to & a \\
B & \to & b \\
Z & \to & 0 \\
O & \to & 1 \\
P & \to & + \\
M & \to & * \\
L & \to & (\\
R & \to &) \\
C_1 & \to & PT \\
C_2 & \to & MF \\
C_3 & \to & ER
\end{array}
$$

Figure 3.2 — Breaking bodies of length 3 with C1 → PT, C2 → MF, C3 → ER (sl. 35)

A body X1X2…Xk (k ≥ 3) becomes A → X1C1, C1 → X2C2, …, C(k−2) → X(k−1)Xk.

Exam Approach

1. Write the original language in one line if easy — it helps you check the result.
2. Do steps 1–3 exactly as in Lesson 2, showing the grammar after each.
3. New variables for terminals (Z → 0, U → 1); new variables for long bodies.
4. Check: every rule is A → BC or A → a (plus S → ε if ε ∈ L); derive one short string.

TRAP Never leave A → ε or A → B in the final grammar; never replace a lone terminal body A → 0 by a variable; keep ε only as S → ε for the (new) start symbol.

Practice — Lesson 3

(a) Actual — 2021-22 Q5 / 2017-18 Q7(a). (b) Actual — 2016-17 Q6(b) / 2014-15 Q5(c).

(a) Convert S → A1A | B, A → 0A | 1A | ε, D → 00 (2017-18; in 2021-22 the last rule is B → 1) into an equivalent grammar in CNF. Show the steps. (8–15)

(b) What is CNF? Convert A → BAB | B | ε, B → 00 | ε into an equivalent grammar in CNF. (9⅓–10)

✓ Solution

(a) L(G) = binary strings containing at least one 1 (S → A1A, A = (0 ∪ 1)∗).

```
Step 1 (ε): nullable A.   S → A1A | 1A | A1 | 1 | B     A → 0A | 0 | 1A | 1     D → 00   (2021-22: B
→ 1)
Step 2 (unit): unit pair (S,B).  2017-18: B has no productions → S → B simply disappears.
                                 2021-22: S → B, B → 1 gives S → 1 (already present).
Step 3 (useless): B is not generating (2017-18); D is not reachable (2017-18). Remove both.
        S → A1A | 1A | A1 | 1        A → 0A | 1A | 0 | 1
Step 4a: Z → 0, U → 1.   S → AUA | UA | AU | 1     A → ZA | UA | 0 | 1
Step 4b: S → AUA becomes S → VA, V → AU.
CNF:  S → VA | UA | AU | 1      V → AU      A → ZA | UA | 0 | 1      Z → 0      U → 1
```

Checked: the CNF grammar generates exactly the strings with at least one 1 (all strings up to length 7).

(b) CNF definition: Lesson 3 box. L = {(00)^k | k ≥ 0} — note ε ∈ L.

```
Step 1 (ε): nullable A, B.  A → BAB | AB | BA | BB | B | A   (drop A → A)     B → 00
           ε ∈ L → new start S0 → A | ε
Step 2 (unit): A → B gives A → 00;  S0 → A gives S0 → (all A bodies)
        S0 → BAB | AB | BA | BB | 00 | ε      A → BAB | AB | BA | BB | 00      B → 00
Step 3 (useless): none.
Step 4a: Z → 0:  …… 00 becomes ZZ
Step 4b: BAB becomes BC with C → AB
CNF:  S0 → BC | AB | BA | BB | ZZ | ε     A → BC | AB | BA | BB | ZZ     C → AB     B → ZZ     Z → 0
```

Checked: the CNF grammar (with S0 → ε) generates exactly (00)∗ up to length 8.

Lesson 4 — The CYK Membership Algorithm

Lecture 2, slides 4–18

CYK (Cocke–Younger–Kasami) decides $w \in L(G)$ for a CNF grammar G in $O(n^3)$ time by dynamic programming (sl. 4). X_{ij} = set of variables A with $A \Rightarrow^* a_i \ldots a_j$ (sl. 8); $w \in L$ iff $S \in X_{1n}$ (sl. 9).

$$X_{ii} = \{ A \mid A \to a_i \}$$
$$A \in X_{ij} \iff \exists k,\ i \le k < j,\ B \in X_{ik},\ C \in X_{k+1,j},\ A \to BC$$

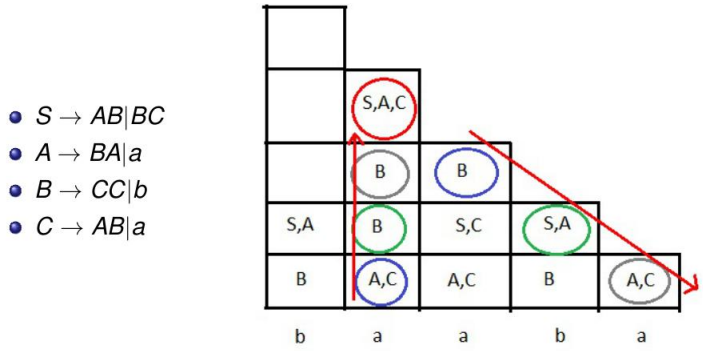


Figure 4.1 — How X_{ij} is computed: pair the cells below and diagonally (sl. 17)

Worked Example: baaba (sl. 11–18)

```
S → AB | BC    A → BA | a    B → CC | b    C → AB | a
```

```
len 5: X15={A,C,S}
len 4: X14=Ø          X25={A,C,S}
len 3: X13=Ø          X24={B}        X35={B}
len 2: X12={A,S}      X23={B}        X34={C,S}      X45={A,S}
len 1: X11={B}        X22={A,C}      X33={A,C}      X44={B}        X55={A,C}
        b              a              a              b              a
S ∈ X15  ⇒  baaba ∈ L(G)
```

Exam Approach

1. Bottom row: for each symbol the variables with A → symbol.
2. Row of length L: for each cell try every split k; for each pair (B in left cell, C in right cell) collect the heads A → BC. Write the pairs you tried in rough work.
3. Top cell: answer 'w ∈ L(G)' iff S is in it.
4. If asked to describe L(G), read the CNF rules backwards into a familiar grammar.

TRAP X_{ij} uses ALL splits k = i … j−1, not only the middle one. A missing variable in a low row propagates upward — double-check the bottom two rows.

Practice — Lesson 4

(a) Actual — 2023-24 Q6, 2019-20 Q7, Jan-2020 Q7. (b) Actual — 2022-23 Q7. (c) Actual — 2017-18 Q7(b). (d) Actual — 2007-08 Q13.

(a) G (CNF): S → BZ | CY | ZZ | YY, B → ZA, C → YA, A → ZA | YA | 0 | 1, Z → 0, Y → 1. Using CYK determine whether 0010 (2023-24), 01100 (2019-20), 10011 (Jan-2020) is in L(G). Describe L(G). (15)

(b) G: S → AB | AY | YA | 1, A → ZA | YA | 0 | 1, B → YA, Z → 0, Y → 1. Is 0100 in L(G)? (15)

(c) Using the CNF grammar of Lesson 3 (a), is 01101 in the language? (7)

(d) CNF definition; CYK for abaab with S → AB | BC, A → BA | a, B → CC | b, C → AB | a. (10)

✓ Solution

(a) A ⇒∗ any non-empty binary string (A → ZA | YA | 0 | 1); B ⇒ 0A, C ⇒ 1A. So S → BZ = 0(0|1)⁺0, CY = 1(0|1)⁺1, ZZ = 00, YY = 11: **L(G) = binary strings of length ≥ 2 whose first and last symbols are equal** (checked by program). Tables:

```
len 4: X14={A,B,S}
len 3: X13={A,B}      X24={A,B,S}
len 2: X12={A,B,S}    X23={A,B}      X34={A,C}
len 1: X11={A,Z}      X22={A,Z}      X33={A,Y}      X44={A,Z}
        0              0              1              0
S ∈ X14  ⇒  0010 ∈ L(G)
```

```
len 5: X15={A,B,S}
len 4: X14={A,B,S}    X25={A,C}
len 3: X13={A,B}      X24={A,C}      X35={A,C}
len 2: X12={A,B}      X23={A,C,S}    X34={A,C}      X45={A,B,S}
len 1: X11={A,Z}      X22={A,Y}      X33={A,Y}      X44={A,Z}      X55={A,Z}
        0              1              1              0              0
S ∈ X15  ⇒  01100 ∈ L(G)
```

```
len 5: X15={A,C,S}
len 4: X14={A,C,S}    X25={A,B}
len 3: X13={A,C}      X24={A,B}      X35={A,B}
len 2: X12={A,C}      X23={A,B,S}    X34={A,B}      X45={A,C,S}
len 1: X11={A,Y}      X22={A,Z}      X33={A,Z}      X44={A,Y}      X55={A,Y}
        1              0              0              1              1
S ∈ X15  ⇒  10011 ∈ L(G)
```

(b)

```
len 4: X14={A,S}
len 3: X13={A,S}      X24={A,B,S}
len 2: X12={A,S}      X23={A,B,S}    X34={A}
len 1: X11={A,Z}      X22={A,S,Y}    X33={A,Z}      X44={A,Z}
        0              1              0              0
S ∈ X14  ⇒  0100 ∈ L(G)
```

L(G): S → AY | YA | AB … gives non-empty strings containing a 1 in a suitable position; CYK only needs the table.

(c) CNF: S → VA | UA | AU | 1, V → AU, A → ZA | UA | 0 | 1, Z → 0, U → 1.

```
len 5: X15={A,S,V}
len 4: X14={A,S}      X25={A,S,V}
len 3: X13={A,S,V}    X24={A,S}      X35={A,S,V}
len 2: X12={A,S,V}    X23={A,S,V}    X34={A,S}      X45={A,S,V}
len 1: X11={A,Z}      X22={A,S,U}    X33={A,S,U}    X44={A,Z}      X55={A,S,U}
        0              1              1              0              1
S ∈ X15  ⇒  01101 ∈ L(G)
```

(d) CNF definition: Lesson 3.

```
len 5: X15={C,S}
len 4: X14={A,S}      X25=∅
len 3: X13={B}        X24=∅          X35={B}
len 2: X12={C,S}      X23={A,S}      X34={B}        X45={C,S}
len 1: X11={A,C}      X22={B}        X33={A,C}      X44={A,C}      X55={B}
        a              b              a              a              b
S ∈ X15  ⇒  abaab ∈ L(G)
```

Quick Notes — Intro & CNF

Revision points for Lessons 1–4.

L1 · Areas

- Automata (models) · computability (solvable?) · complexity (efficiently?).
- FA ⊂ CFG/PDA ⊂ TM in power; TM simulates a computer.

L2 · Simplification

- Useless = not generating or not reachable; remove generating-failures first.
- Nullable: add all versions with nullable symbols dropped; keep S → ε only if ε ∈ L.
- Unit pairs (A, B): copy B's non-unit bodies to A; delete unit rules.

L3 · CNF

- A → BC | a, no useless symbols (+ S → ε).
- Order: ε → unit → useless → terminals → break long bodies.
- S → A1A | B, A → 0A|1A|ε ⇒ S → VA|UA|AU|1, V → AU, A → ZA|UA|0|1.
- A → BAB|B|ε, B → 00|ε ⇒ (00)∗; S0 → BC|AB|BA|BB|ZZ|ε, C → AB, B → ZZ.

L4 · CYK

- X_{ii} from A → a; X_{ij} from all splits; w ∈ L iff S ∈ X_{1n}; O(n³).
- S → BZ|CY|ZZ|YY grammar = length ≥ 2, first = last symbol (asked 3×).

Key Diagrams to Reproduce

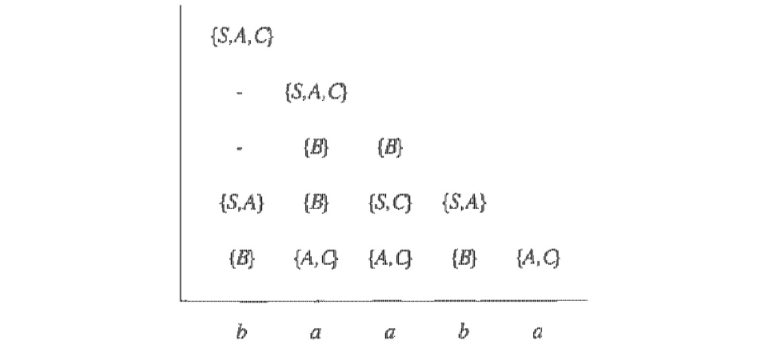


Figure 7.14: The table for string *baaba* constructed by the CYK algorithm

CYK table for baaba (sl. 18)